

Laxmi Charitable Trust's
Sheth L.U.J College of Arts & Sir M.V. College of Science and Commerce
Department of Information Technology (B.Sc.I.T Semester V)
Dot .Net Core and Progressive Web Application Practical

Practical – III

Roll No.: T003	Name: Pranay Bhadwalkar
Class: TYIT	Batch: B1
Date of Assignment: 14/07/2026	Date/Time of Submission: 14/07/2026

Part A - Theory :

C# Delegate

What is a Delegate?

A **Delegate** is a special type in C# that stores the reference (address) of a method. It allows a method to be called indirectly through a delegate object. A delegate can call any method that has the **same return type and parameters** as the delegate declaration.

Why are Delegates Used?

- To call methods indirectly.
- To increase code flexibility.
- To implement callbacks.
- Used in event handling and GUI programming.

Real-Life Example

Think of a **TV Remote**.

- TV = Method
- Remote = Delegate

Instead of pressing buttons on the TV, we use the remote. Similarly, instead of calling a method directly, we use a delegate.

Syntax of Delegate

```
delegate returnType DelegateName(parameter_list);
```

Example

```
delegate void Operation(int a, int b);
```

Steps to Use a Delegate

Step 1: Declare the delegate

```
delegate void Operation(int a, int b);
```

Step 2: Create methods having the same signature

```
public void Add(int a, int b)
{
    Console.WriteLine(a + b);
}
```

Step 3: Create a delegate object

```
Operation obj;
```

Step 4: Assign a method to the delegate

```
obj = Add;
```

Step 5: Invoke the delegate

```
obj(10, 20);
```

or

```
obj.Invoke(10, 20);
```

Part B - Program

- a) Create methods `add()`, `multiply()`, `subtract()`, `divide()` with suitable parameters and call these methods using concept of C# delegate.

Algorithm:

- Step 1: Start the program.
- Step 2: Declare a delegate `Operation`.
- Step 3: Create the `Calculator` class.
- Step 4: Define the methods `Add()`, `Sub()`, `Multiply()`, and `Div()`.
- Step 5: Create an object of the `Calculator` class.
- Step 6: Declare a delegate object `obj`.
- Step 7: Assign the `Add()` method to the delegate and call it.
- Step 8: Assign the `Sub()` method to the delegate and call it.
- Step 9: Assign the `Multiply()` method to the delegate and call it.
- Step 10: Assign the `Div()` method to the delegate and call it.
- Step 11: Display the results.
- Step 12: Stop the program.

Code:

```
using System;

delegate void Operation(int a, int b);

class Calculator
{
    public void Add(int a, int b)
    {
        Console.WriteLine("Addition = " + (a + b));
    }

    public void Sub(int a, int b)
    {
        Console.WriteLine("Subtraction = " + (a - b));
    }

    public void Multiply(int a, int b)
    {
        Console.WriteLine("Multiplication = " + (a * b));
    }

    public void Div(int a, int b)
    {
        Console.WriteLine("Division = " + (a / b));
    }
}
```

```

class Program
{
    static void Main()
    {
        Calculator c = new Calculator();
        Operation obj;

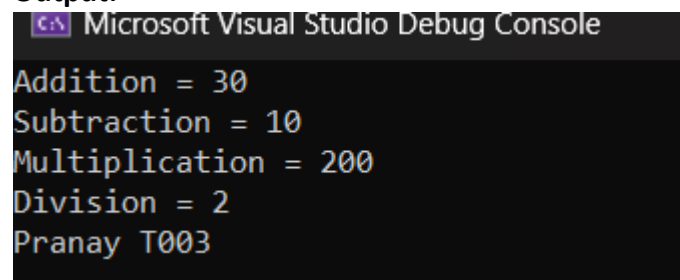
        obj = c.Add;
        obj(20, 10);

        obj = c.Sub;
        obj(20, 10);

        obj = c.Multiply;
        obj(20, 10);

        obj = c.Div;
        obj(20, 10);
        Console.WriteLine("Pranay T003");
    }
}

```

Output:


```

c:\> Microsoft Visual Studio Debug Console
Addition = 30
Subtraction = 10
Multiplication = 200
Division = 2
Pranay T003

```

b) Write a program using multicast delegate.**Algorithm:**

- Step 1: Start the program.
- Step 2: Declare a delegate Message.
- Step 3: Create the Demo class.
- Step 4: Define the methods Welcome(), Learn(), and ThankYou().
- Step 5: Create an object of the Demo class.
- Step 6: Declare a delegate object obj.
- Step 7: Assign the Welcome() method to the delegate.
- Step 8: Add the Learn() and ThankYou() methods to the delegate.
- Step 9: Invoke the delegate to execute all three methods.
- Step 10: Display "Pranay T003".
- Step 11: Stop the program.

Code:

```
using System;

delegate void Message();

class Demo
{
    public void Welcome()
    {
        Console.WriteLine("Welcome Students");
    }

    public void Learn()
    {
        Console.WriteLine("Learn Delegates");
    }

    public void ThankYou()
    {
        Console.WriteLine("Thank You");
    }
}

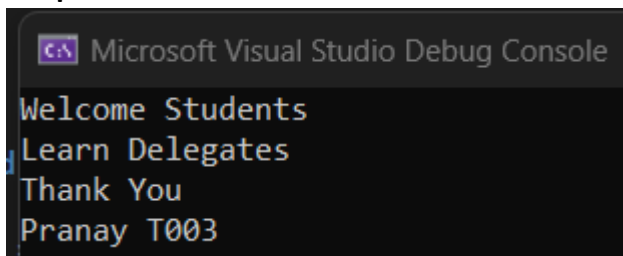
class Program
{
    static void Main()
    {
        Demo d = new Demo();

        Message obj;

        obj = d.Welcome;
        obj += d.Learn;
        obj += d.ThankYou;

        obj();

        Console.WriteLine("Pranay T003");
    }
}
```

Output:

```
Microsoft Visual Studio Debug Console
Welcome Students
Learn Delegates
Thank You
Pranay T003
```

- c) Create a class BankAccount with AccountNumber and Balance. Implement property for Balance with validation (Balance cannot be negative).

Algorithm:

- Step 1: Start the program.
- Step 2: Create the BankAccount class.
- Step 3: Declare the AccountNumber property.
- Step 4: Declare the private variable balance.
- Step 5: Create the Balance property with get and set methods.
- Step 6: Check if the balance is greater than or equal to 0 before assigning it.
- Step 7: Create an object of the BankAccount class.
- Step 8: Assign values to AccountNumber and Balance.
- Step 9: Display the account number and balance.
- Step 10: Try to assign a negative balance and display the error message.
- Step 11: Display "Pranay T003".
- Step 12: Stop the program.

Code:

```
using System;

class BankAccount
{
    public int AccountNumber { get; set; }

    private double balance;

    public double Balance
    {
        get
        {
            return balance;
        }
        set
        {
            if (value >= 0)
                balance = value;
            else
                Console.WriteLine("Balance cannot be negative.");
        }
    }
}

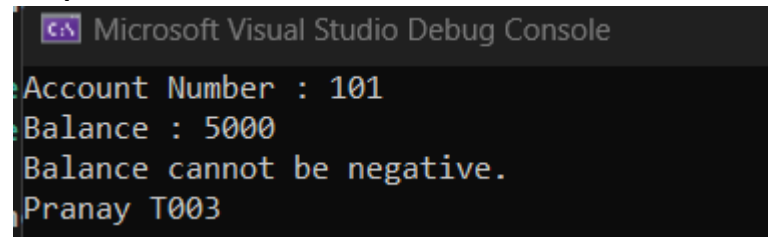
class Program
{
    static void Main()
    {
        BankAccount b = new BankAccount();

        b.AccountNumber = 101;
        b.Balance = 5000;

        Console.WriteLine("Account Number : " + b.AccountNumber);
        Console.WriteLine("Balance : " + b.Balance);

        b.Balance = -1000;
        Console.WriteLine("Pranay T003");
    }
}
```

```
}  
}
```

Output:A screenshot of the Microsoft Visual Studio Debug Console. The title bar reads "Microsoft Visual Studio Debug Console". The console output shows four lines: "Account Number : 101", "Balance : 5000", "Balance cannot be negative.", and "Pranay T003".

```
Account Number : 101  
Balance : 5000  
Balance cannot be negative.  
Pranay T003
```

Part C – Curiosity Question

1. A college wants to **calculate student performance**.
 - a. Create methods to
 - b. Calculate Total Marks
 - c. Calculate Percentage
 - d. Display Grade
 - e. Display Pass/Fail

Note: Call each method using a delegate.

Algorithm:

Step 1: Start the program.

Step 2: Declare a delegate StudentDelegate.

Step 3: Create the Student class.

Step 4: Define the methods CalculateTotal(), CalculatePercentage(), DisplayGrade(), and DisplayResult().

Step 5: Create an object of the Student class.

Step 6: Declare a delegate object obj.

Step 7: Assign CalculateTotal() to the delegate.

Step 8: Add CalculatePercentage(), DisplayGrade(), and DisplayResult() to the delegate.

Step 9: Invoke the delegate to execute all the methods.

Step 10: Display "Pranay T003".

Step 11: Stop the program.

Code:

```
using System;

delegate void StudentDelegate();

class Student
{
    int total;
    double percentage;

    public void CalculateTotal()
    {
        int m1 = 70, m2 = 80, m3 = 90;
        total = m1 + m2 + m3;
        Console.WriteLine("Total Marks: " + total);
    }

    public void CalculatePercentage()
    {
        percentage = total / 3.0;
        Console.WriteLine("Percentage: " + percentage);
    }

    public void DisplayGrade()
    {
        if (percentage >= 75)
```



```

        Console.WriteLine("Grade: A");
    else if (percentage >= 60)
        Console.WriteLine("Grade: B");
    else if (percentage >= 50)
        Console.WriteLine("Grade: C");
    else
        Console.WriteLine("Grade: Fail");
    }

    public void DisplayResult()
    {
        if (percentage >= 50)
            Console.WriteLine("Result: Pass");
        else
            Console.WriteLine("Result: Fail");
    }
}

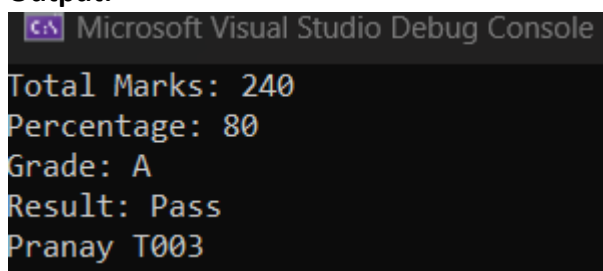
class Program
{
    static void Main()
    {
        Student s = new Student();

        StudentDelegate obj;

        obj = s.CalculateTotal;
        obj += s.CalculatePercentage;
        obj += s.DisplayGrade;
        obj += s.DisplayResult;

        obj();
        Console.WriteLine("Pranay T003");
    }
}

```

Output:


```

Microsoft Visual Studio Debug Console
Total Marks: 240
Percentage: 80
Grade: A
Result: Pass
Pranay T003

```

2. A college wants to automate the admission process. When the user clicks **Confirm Admission**, several operations must happen automatically. Design a delegate-based solution.

Students decide:

- a. Verify Documents
- b. Generate Roll Number
- c. Allocate Division
- d. Send SMS

Note: For above operation just print a statement.

Algorithm:

Step 1: Start the program.

Step 2: Declare a delegate AdmissionDelegate.

Step 3: Create the Admission class.

Step 4: Define the methods VerifyDocuments(), GenerateRollNumber(), AllocateDivision(), and SendSMS().

Step 5: Create an object of the Admission class.

Step 6: Declare a delegate object obj.

Step 7: Assign VerifyDocuments() to the delegate.

Step 8: Add GenerateRollNumber(), AllocateDivision(), and SendSMS() to the delegate.

Step 9: Display the message "Confirm Admission Clicked...".

Step 10: Invoke the delegate to execute all the methods.

Step 11: Display "Pranay T003".

Step 12: Stop the program.

Code:

```
using System;

delegate void AdmissionDelegate();

class Admission
{
    public void VerifyDocuments()
    {
        Console.WriteLine("Documents Verified");
    }

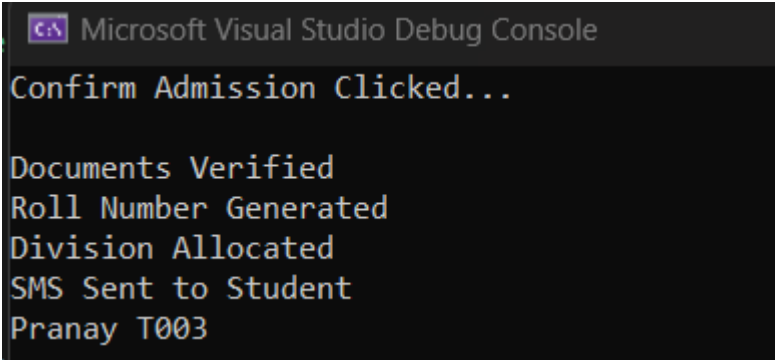
    public void GenerateRollNumber()
    {
        Console.WriteLine("Roll Number Generated");
    }

    public void AllocateDivision()
    {
        Console.WriteLine("Division Allocated");
    }

    public void SendSMS()
    {
        Console.WriteLine("SMS Sent to Student");
    }
}

class Program
{
    static void Main()
    {
        Admission a = new Admission();
```

```
AdmissionDelegate obj;  
  
obj = a.VerifyDocuments;  
obj += a.GenerateRollNumber;  
obj += a.AllocateDivision;  
obj += a.SendSMS;  
  
Console.WriteLine("Confirm Admission Clicked...\n");  
  
obj();  
  
Console.WriteLine("Pranay T003");  
}  
}
```

Output:

Microsoft Visual Studio Debug Console

```
Confirm Admission Clicked...  
  
Documents Verified  
Roll Number Generated  
Division Allocated  
SMS Sent to Student  
Pranay T003
```